KEMENTERIAN PENDIDIKAN TINGGI, SAINS, DAN TEKNOLOGI

UNIVERSITAS MARITIM RAJA ALI HAJI

FAKULTAS TEKNIK DAN TEKNOLOGI KEMARITIMAN

PROGRAM STUDI S1 TEKNIK INFORMATIKA

Penambangan Data - Minggu 10 - 2401020073 - Nabil

TUGAS MENULIS INDIVIDU: KAJIAN MENDALAM TEKNIK KLASIFIKASI

Formulasi Teoretis, Algoritma Hunt, ID3, C4.5, CART, Naïve Bayes, SVM, Regresi Logistik, Neural Network, Simulasi Hitung Manual 14 Kasus Pasien, dan Evaluasi Model

Disusun Oleh:

NABIL

NIM: 2401020073

Dosen Pengampu Mata Kuliah:

1. Tekad Matulatan, S.T., M.Kom.

2. Muhamad Fadli, S.T., M.Kom.

TANJUNGPINANG, KEPULAUAN RIAU

2026

BAB I
PENGANTAR DAN KONSEP SUPERVISED LEARNING

1.1 Definisi dan Formalisasi Masalah Klasifikasi

Dalam data mining dan machine learning (Han et al., 2012), klasifikasi adalah salah satu tugas utama pada supervised learning. Secara matematis, masalah klasifikasi bisa ditulis sebagai pemetaan dari vektor input x yang berada di ruang fitur berdimensi d ($x \in X \subseteq R^d$) ke sebuah label kelas y yang bersifat diskrit, $y \in Y = \{c1, c2, ..., cK\}$, dengan $K \geq 2$ adalah jumlah kelas. Setiap kelas saling lepas (mutually exclusive).

Misalkan kita punya data latih (training dataset) berisi N pasang data yang sudah berlabel:

$$D = \{(x1, y1), (x2, y2), ..., (xN, yN)\}, \quad \text{dengan } (xi, yi) \sim P(X, Y) \text{ i.i.d.} \quad \textit{(Persamaan 1.1)}$$

Tujuan algoritma klasifikasi adalah mencari fungsi hipotesis $f: X \rightarrow Y$ yang meminimalkan kesalahan prediksi pada data yang belum pernah dilihat (unseen data). Jadi model yang bagus bukan model yang hafal data latih, melainkan model yang bisa generalisasi dengan baik (Mitchell, 1997). Model yang cuma hafal biasanya jelek saat dipakai pada data baru.

1.2 Klasifikasi vs Regresi vs Clustering

Supaya tidak tertukar (Tan et al., 2019), perbedaan ketiganya adalah sebagai berikut:

1. Klasifikasi (Supervised Learning, target diskrit): model memisahkan data ke dalam kelas yang sudah ditentukan. Contohnya deteksi email spam atau bukan spam, diagnosis penyakit jantung positif atau negatif, dan deteksi transaksi bank yang curang.
2. Regresi (Supervised Learning, target kontinu): targetnya berupa bilangan real, $y \in R$. Contohnya prediksi jumlah penjualan obat per bulan di puskesmas, perkiraan curah hujan, dan prediksi harga saham.

3. Clustering (Unsupervised Learning, tanpa label): data tidak punya label y. Algoritma mengelompokkan data berdasarkan kemiripan, misalnya jarak Euclidean. Contohnya segmentasi pasien berdasarkan demografi atau pengelompokan pola konsumsi obat.

1.3 Pembagian Dataset: Training, Validation, dan Testing

Pembagian data itu penting supaya tidak terjadi data leakage dan hasil evaluasi tidak terlalu optimis. Pembagian yang umum dipakai adalah:

- Training Set (60%-70%): dipakai untuk melatih parameter model, misalnya memilih pemisah pada node pohon keputusan atau menyesuaikan bobot jaringan saraf.
- Validation Set (15%-20%): dipakai untuk mengatur hyperparameter, misalnya pruning threshold, nilai C pada SVM, atau jumlah neuron tersembunyi, sekaligus untuk mengecek overfitting.
- Testing Set (15%-20%): dipakai sekali di akhir untuk mengukur performa model pada data yang belum pernah dipakai sebelumnya.

1.4 Inductive Bias dan No Free Lunch Theorem

Menurut No Free Lunch Theorem (Wolpert & Macready, 1997), tidak ada satu algoritma machine learning yang paling bagus untuk semua jenis data. Setiap algoritma punya inductive bias sendiri. Decision tree, misalnya, mengasumsikan batas keputusan sejajar sumbu. Naïve Bayes mengasumsikan atribut saling independen jika kelasnya sudah diketahui. SVM linear mengasumsikan data bisa dipisahkan secara linear dengan margin maksimal. Karena itu kita perlu paham beberapa algoritma sekaligus supaya bisa memilih yang cocok dengan kasusnya.

BAB II
KELUARGA ALGORITMA DECISION TREE

2.1 Konsep Dasar dan Algoritma Hunt

Decision tree adalah struktur graf berarah tanpa siklus yang menyusun aturan if-then secara bertingkat. Strukturnya terdiri atas tiga bagian:

- Root node: simpul paling atas, berisi atribut pemilah pertama, yaitu atribut yang paling baik memisahkan data.
- Internal node: simpul yang berisi uji kondisi pada nilai atribut tertentu.
- Leaf node: simpul paling akhir yang berisi keputusan kelas.

Hampir semua algoritma decision tree (ID3, C4.5, dan CART) dibangun di atas Algoritma Hunt (1966). Misalkan Dt adalah himpunan data latih yang sampai di simpul t. Langkahnya sebagai berikut:

1. Kalau semua data di Dt punya label kelas yang sama (murni), simpul t dijadikan leaf dengan label tersebut.
2. Kalau Dt kosong, simpul t dijadikan leaf dengan label kelas mayoritas dari simpul induknya.
3. Kalau Dt masih berisi lebih dari satu kelas, pilih atribut terbaik berdasarkan ukuran impurity (misalnya entropy atau gini), pecah data sesuai atribut itu, lalu ulangi proses untuk tiap subset secara rekursif.

2.2 Algoritma ID3: Shannon Entropy dan Information Gain

ID3 (Iterative Dichotomiser 3) dikembangkan oleh J. Ross Quinlan (1986). Algoritma ini memakai konsep entropy dari teori informasi Shannon (1948) untuk menentukan atribut pemisah terbaik.

Entropy mengukur seberapa acak atau tidak murninya distribusi kelas pada himpunan data S:

$$H(S) = -\sum [\, p_i * \log2(p_i) \,], \quad \text{untuk } i = 1 \text{ sampai } K \qquad \textit{(Persamaan 2.1)}$$

p_i adalah proporsi sampel kelas c_i di dalam S. Kalau semua data di S berasal dari satu kelas, H(S) = 0. Entropy paling besar terjadi saat data terbagi rata. Untuk kasus biner 50:50, nilainya 1 bit.

Information gain (IG) mengukur seberapa besar penurunan entropy setelah S dipecah berdasarkan atribut A:

> $IG(S, A) = H(S) - \sum [(|S_v| / |S|) * H(S_v)]$, untuk setiap $v \in Values(A)$ *(Persamaan 2.2)*

ID3 memilih atribut A yang menghasilkan IG(S, A) paling besar. Masalahnya, ID3 cenderung memilih atribut yang punya banyak nilai unik, seperti ID pasien atau nomor KTP. Atribut seperti itu memang membuat tiap cabang murni, tapi tidak punya kemampuan generalisasi sama sekali.

2.3 Algoritma C4.5: Atribut Kontinu, Missing Values, dan Gain Ratio

C4.5 dikembangkan oleh Quinlan (1993) sebagai perbaikan dari ID3. Ada tiga perbaikan utama:

1. Gain ratio dan split information. IG dinormalisasi dengan split information dari atribut itu sendiri:

   > $SplitInfo_A(S) = - \sum [(|S_v| / |S|) * \log2(|S_v| / |S|)]$ *(Persamaan 2.3)*

   > $GainRatio(S, A) = IG(S, A) / SplitInfo_A(S)$ *(Persamaan 2.4)*

   Atribut dengan nilai unik yang banyak akan punya SplitInfo yang besar, sehingga GainRatio-nya ikut turun. Dengan begitu bias ID3 ke atribut seperti itu bisa dikurangi.

2. Atribut kontinu. Nilainya diurutkan dulu, lalu dicari titik tengah antara dua nilai yang berurutan sebagai kandidat threshold biner. Threshold yang menghasilkan IG paling besar yang dipilih.
3. Missing values. Data yang nilai atributnya kosong dibagi ke semua cabang dengan bobot sesuai proporsi nilai yang ada.

2.4 Algoritma CART: Gini Impurity dan Binary Splitting

CART (Classification and Regression Trees) dikembangkan oleh Breiman et al. (1984). Ciri utamanya:

- Pohonnya selalu biner, jadi setiap simpul hanya punya dua anak (True/False, atau $\leq \theta$ dan $> \theta$).
- Memakai ukuran impurity Gini:

> **Gini(S) = 1 - ∑ [(p_i)^2], untuk i = 1 sampai K** *(Persamaan 2.5)*

Kalau simpulnya murni, Gini(S) = 0. Kalau datanya terbagi 50:50, Gini(S) = 1 - (0.5^2 + 0.5^2) = 0.50. CART memilih pemisahan biner yang menghasilkan weighted Gini split paling kecil.

2.5 Overfitting dan Pruning

Decision tree yang dibiarkan tumbuh tanpa batas biasanya hafal semua data latih termasuk noise-nya, sehingga performanya jelek pada data baru. Ada dua cara untuk mengatasinya:

1. Pre-pruning (early stopping): pertumbuhan pohon dihentikan lebih awal kalau kedalaman sudah mencapai max_depth, jumlah sampel di simpul kurang dari min_samples_split, atau penurunan impurity lebih kecil dari batas tertentu.
2. Post-pruning (cost-complexity pruning): pohon dibiarkan tumbuh penuh dulu, lalu cabangnya dipangkas dari bawah ke atas dengan kriteria berikut:

> **R_α(T) = R(T) + α * |T|** *(Persamaan 2.6)*

R(T) adalah tingkat kesalahan klasifikasi pohon, |T| adalah jumlah daun, dan $\alpha \geq 0$ adalah parameter penalti kompleksitas yang biasanya dicari dengan cross-validation.

BAB III
KLASIFIKASI PROBABILISTIK DENGAN NAÏVE BAYES

3.1 Teorema Bayes dan Maximum A Posteriori (MAP)

Naïve Bayes bekerja berdasarkan probabilitas Bayesian. Misalkan x = (x1, x2, ..., xd)^T adalah fitur dari sebuah objek. Probabilitas posterior bahwa objek itu masuk kelas c_k dihitung dengan Teorema Bayes:

$$P(y = c_k \mid x) = [\, P(x \mid y = c_k) * P(y = c_k) \,] \,/\, P(x) \quad \textit{(Persamaan 3.1)}$$

Keterangan:

- P(y = c_k | x): posterior, yaitu keyakinan terhadap kelas setelah melihat data.
- P(x | y = c_k): likelihood, yaitu seberapa mungkin pola fitur x muncul di kelas c_k.
- P(y = c_k): prior, yaitu proporsi kelas tersebut di data latih sebelum melihat data baru.
- P(x): evidence, yaitu probabilitas total dari x. Nilainya sama untuk semua kelas, jadi bisa diabaikan saat membandingkan kelas.
- Pada kriteria Maximum A Posteriori (MAP), kelas yang dipilih adalah kelas dengan pembilang paling besar:

$$y_pred = \text{argmax_\{c_k\}} \, [\, P(x \mid y = c_k) * P(y = c_k) \,] \quad \textit{(Persamaan 3.2)}$$

3.2 Asumsi Conditional Independence dan Log-Likelihood

Menghitung probabilitas gabungan P(x1, ..., xd | c_k) secara langsung butuh data yang sangat banyak (curse of dimensionality). Karena itu Naïve Bayes mengasumsikan semua fitur saling independen jika kelasnya sudah diketahui:

$$P(x \mid y = c_k) = \prod [\, P(x_j \mid y = c_k) \,], \quad \text{untuk } j = 1 \text{ sampai } d \quad \textit{(Persamaan 3.3)}$$

Perkalian banyak angka kecil bisa menyebabkan underflow pada floating point. Maka perhitungannya biasanya dilakukan dalam bentuk logaritma natural:

> $y_pred = \text{argmax}_{c_k} [\ln P(y = c_k) + \sum \ln P(x_j \mid y = c_k)]$ *(Persamaan 3.4)*

3.3 Masalah Probabilitas Nol dan Laplace Smoothing

Kalau nilai fitur xj = v tidak pernah muncul di data latih untuk kelas c_k, maka P(v | c_k) = 0. Akibatnya seluruh perkalian jadi 0 dan fitur lain tidak berpengaruh lagi. Solusinya adalah Laplace correction (additive smoothing):

> $P(x_j = v \mid y = c_k) = [\text{count}(x_j = v, y = c_k) + \alpha] / [\text{count}(y = c_k) + \alpha * K_j]$
> *(Persamaan 3.5)*

$\alpha > 0$ adalah parameter smoothing (biasanya $\alpha = 1$), dan K_j adalah jumlah nilai unik pada atribut ke-j.

3.4 Varian Naïve Bayes

- Gaussian Naïve Bayes: untuk data kontinu yang diasumsikan berdistribusi normal $N(\mu, \sigma^2)$.
- Multinomial Naïve Bayes: biasa dipakai untuk teks, berdasarkan frekuensi kemunculan kata.
- Bernoulli Naïve Bayes: untuk fitur biner, misalnya ada atau tidaknya suatu kata atau gejala.

BAB IV
SUPPORT VECTOR MACHINE (SVM)

4.1 Hyperplane dan Margin Maksimal

SVM (Cortes & Vapnik, 1995) berpijak pada prinsip Structural Risk Minimization. Di ruang fitur berdimensi d, bidang pemisah (hyperplane) dinyatakan dengan:

$$w^T * x + b = 0 \quad \textit{(Persamaan 4.1)}$$

w adalah vektor bobot dan b adalah bias. Margin adalah jarak tegak lurus dari bidang pemisah ke titik data terdekat di tiap kelas. SVM mencari bidang yang membuat margin paling lebar:

$$\text{Lebar Margin } \gamma = 2 / ||w|| \quad \textit{(Persamaan 4.2)}$$

4.2 Formulasi Optimasi Hard-Margin dan Dual Lagrange

Memaksimalkan 2/||w|| sama saja dengan meminimalkan 1/2 $||w||^2$. Bentuk primal SVM-nya:

$$\min_{w, b} [(1/2) * ||w||^2] \quad \text{dengan syarat: } y_i * (w^T * x_i + b) \geq 1, \forall i \quad \textit{(Persamaan 4.3)}$$

Dengan Wolfe Duality dan kondisi KKT (Karush-Kuhn-Tucker), masalah ini diubah ke bentuk dual yang hanya memakai dot product antar data:

$$\max_{\alpha} [\sum \alpha_i - (1/2) * \sum \sum \alpha_i * \alpha_j * y_i * y_j * (x_i^T * x_j)] \quad \textit{(Persamaan 4.4)}$$

Titik data yang punya $\alpha_i > 0$ disebut support vectors. Hanya titik-titik inilah yang menentukan posisi hyperplane akhir.

4.3 Soft-Margin SVM dan Parameter C

Data nyata biasanya punya noise atau outlier, jadi dipakai slack variable $\xi_i \geq 0$ supaya beberapa titik boleh melanggar margin:

$$\min_{w, b, \xi} \left[(1/2) * ||w||^2 + C * \sum \xi_i \right] \quad \text{dengan syarat: } y_i * (w^T * x_i + b) \geq 1 - \xi_i \quad \textit{(Persamaan 4.5)}$$

Parameter C mengatur trade-off antara bias dan variance. C besar berarti kesalahan dihukum berat, margin jadi sempit, dan risiko overfitting naik. C kecil membuat margin lebih lebar dan model lebih toleran terhadap noise.

4.4 Kernel Trick untuk Data Non-Linier

Kalau data tidak bisa dipisahkan secara linear di ruang asal R^d, kernel trick memetakan data ke ruang yang berdimensi lebih tinggi lewat fungsi Φ(x), tanpa perlu menghitung koordinatnya secara eksplisit:

$$K(x_i, x_j) = < \Phi(x_i), \Phi(x_j) > \quad \textit{(Persamaan 4.6)}$$

Beberapa kernel yang umum dipakai:

- Linear: K(x, z) = x^T * z
- Polynomial: K(x, z) = (γ * x^T * z + r)^d
- RBF (Gaussian): K(x, z) = exp(-γ * ||x - z||²)

BAB V
REGRESI LOGISTIK DAN JARINGAN SARAF TIRUAN (ANN)

5.1 Regresi Logistik sebagai Model Linear Probabilistik

Regresi logistik adalah model klasifikasi linear yang memperkirakan probabilitas kelas biner memakai fungsi sigmoid:

$$\sigma(z) = 1 / [\, 1 + e^{(-z)} \,], \quad \text{di mana } z = w^{T} * x + b \qquad \textit{(Persamaan 5.1)}$$

Fungsi sigmoid memetakan nilai z dari -∞ sampai +∞ ke interval probabilitas [0, 1]. Turunannya juga sederhana, yaitu dσ(z)/dz = σ(z)(1 - σ(z)), dan ini yang membuat perhitungannya cepat.

5.2 Log-Odds dan Binary Cross-Entropy Loss

Odds didefinisikan sebagai P / (1 - P). Kalau diambil logaritmanya (logit), hubungannya menjadi linear: ln(P / (1 - P)) = w^T * x + b.

Parameter model dicari dengan memaksimalkan log-likelihood, yang ekuivalen dengan meminimalkan Binary Cross-Entropy Loss (fungsi loss ini convex):

$$J(w, b) = - (1/N) * \sum [\, y_i * \ln(y_pred_i) + (1 - y_i) * \ln(1 - y_pred_i) \,] \qquad \textit{(Persamaan 5.2)}$$

5.3 Perceptron dan Keterbatasan XOR

Perceptron tunggal (Rosenblatt, 1958) hanya bisa memisahkan data yang linearly separable. Minsky dan Papert (1969) menunjukkan bahwa perceptron tunggal tidak bisa menyelesaikan fungsi XOR, karena batas keputusannya harus non-linear.

5.4 Multi-Layer Perceptron (MLP) dan Fungsi Aktivasi

MLP mengatasi masalah XOR dengan menambahkan hidden layer di antara input dan output. Cybenko (1989) membuktikan bahwa MLP dengan fungsi

aktivasi non-linear bisa mendekati fungsi kontinu apa pun (universal approximation theorem).

Beberapa fungsi aktivasi yang sering dipakai:

- ReLU: f(z) = max(0, z). Membantu mengurangi masalah vanishing gradient untuk nilai positif.
- Tanh: f(z) = tanh(z). Outputnya berpusat di nol (zero-centered) dengan rentang [-1, 1].
- Softmax: mengubah output menjadi distribusi probabilitas, biasanya dipakai pada klasifikasi multi-kelas.

5.5 Pembelajaran Jaringan Saraf dengan Backpropagation

Bobot jaringan saraf dilatih dengan backpropagation (Rumelhart et al., 1986). Dengan aturan rantai (chain rule), gradien error dihitung dari layer output lalu dirambatkan mundur ke hidden layer. Setelah itu bobot diperbarui dengan gradient descent:

$$w_{jk} \leftarrow w_{jk} - \eta * (\partial E / \partial w_{jk}) \quad \textit{(Persamaan 5.3)}$$

BAB VI
SIMULASI PERHITUNGAN MANUAL

6.1 Dataset Simulasi: Diagnosa Penyakit Jantung 14 Pasien

Untuk menunjukkan pemahaman saya tentang konsep di atas, bagian ini berisi perhitungan manual langkah demi langkah pada contoh data rekam medis 14 pasien kardiologi.

Tabel 6.1: Data Rekam Medis 14 Pasien untuk Simulasi

ID	Usia (X1)	Tekanan Darah (X2)	Kolesterol (X3)	Riwayat Keluarga (X4)	Penyakit Jantung (Y)
P01	Muda	Tinggi	Tinggi	Tidak	Negatif (N)
P02	Muda	Tinggi	Tinggi	Ya	Negatif (N)
P03	Paruh_Baya	Tinggi	Tinggi	Tidak	Positif (P)
P04	Lanjut	Normal	Tinggi	Tidak	Positif (P)
P05	Lanjut	Normal	Normal	Tidak	Positif (P)
P06	Lanjut	Normal	Normal	Ya	Negatif (N)
P07	Paruh_Baya	Normal	Normal	Ya	Positif (P)
P08	Muda	Normal	Tinggi	Tidak	Negatif (N)
P09	Muda	Normal	Normal	Tidak	Positif (P)
P10	Lanjut	Normal	Tinggi	Tidak	Positif (P)
P11	Muda	Normal	Normal	Ya	Positif (P)
P12	Paruh_Baya	Normal	Tinggi	Ya	Positif (P)
P13	Paruh_Baya	Tinggi	Normal	Tidak	Positif (P)
P14	Lanjut	Normal	Tinggi	Ya	Negatif (N)

Dataset ini berisi 14 pasien dengan 4 atribut prediktor kategorikal (Usia, Tekanan Darah, Kolesterol, Riwayat Keluarga) dan satu label target biner (Penyakit Jantung: Positif atau Negatif). Dari 14 data, ada 9 pasien Positif dan 5 pasien Negatif.

6.2 Perhitungan Manual Decision Tree ID3

Langkah 1: Menghitung entropy awal H(S)

Proporsi kelas: p(+) = 9/14 ≈ 0.642857; p(-) = 5/14 ≈ 0.357143.

$$H(S) = - [(9/14) * \log2(9/14) + (5/14) * \log2(5/14)] = - [-0.409776 - 0.530509] =$$

> **0.940285 bit**

Langkah 2: Menghitung entropy dan information gain untuk tiap atribut

1. Atribut Usia (Muda [5], Paruh_Baya [4], Lanjut [5]):
   - Muda (2 Positif, 3 Negatif): H = -[(2/5) log2(2/5) + (3/5) log2(3/5)] = 0.970951 bit.
   - Paruh_Baya (4 Positif, 0 Negatif): semua datanya positif, jadi H = 0.
   - Lanjut (3 Positif, 2 Negatif): H = -[(3/5) log2(3/5) + (2/5) log2(2/5)] = 0.970951 bit.

   Entropy rata-rata: H(S, Usia) = (5/14)(0.970951) + (4/14)(0) + (5/14)(0.970951) = 0.693536 bit.

   Information gain: IG(S, Usia) = 0.940285 - 0.693536 = 0.246749.

2. Atribut Tekanan Darah (Tinggi [4], Normal [10]):
   - Tinggi (2 Positif, 2 Negatif): H = 1.000000 bit.
   - Normal (7 Positif, 3 Negatif): H = 0.881291 bit.

   Entropy rata-rata: H(S, Tekanan) = (4/14)(1.000000) + (10/14)(0.881291) = 0.915208 bit.

   Information gain: IG(S, Tekanan) = 0.940285 - 0.915208 = 0.025077.

3. Atribut Kolesterol (Tinggi [8], Normal [6]):
   - Tinggi (4 Positif, 4 Negatif): H = 1.000000 bit.
   - Normal (5 Positif, 1 Negatif): H = 0.650022 bit.

   Entropy rata-rata: H(S, Kolesterol) = (8/14)(1.000000) + (6/14)(0.650022) = 0.850010 bit.

   Information gain: IG(S, Kolesterol) = 0.940285 - 0.850010 = 0.090275.

4. Atribut Riwayat Keluarga (Ya [6], Tidak [8]):
   - Ya (3 Positif, 3 Negatif): H = 1.000000 bit.
   - Tidak (6 Positif, 2 Negatif): H = 0.811278 bit.

   Entropy rata-rata: H(S, Riwayat) = (6/14)(1.000000) + (8/14)(0.811278) = 0.892158 bit.

Information gain: IG(S, Riwayat) = 0.940285 - 0.892158 = 0.048127.

Tabel 6.2: Rekapitulasi Information Gain Iterasi 1 (Penentuan Root Node)

Peringkat	Atribut Prediktor	Entropi Kondisional H(S, A)	Information Gain IG(S, A)	Status Keputusan
1	Usia	0.693536 bit	0.246749	TERPILIH SEBAGAI ROOT NODE
2	Kolesterol	0.850010 bit	0.090275	Dieliminasi pada iterasi 1
3	Riwayat Keluarga	0.892158 bit	0.048127	Dieliminasi pada iterasi 1
4	Tekanan Darah	0.915208 bit	0.025077	Dieliminasi pada iterasi 1

Dari tabel ini terlihat bahwa Usia punya IG paling besar (0.246749), jauh di atas Kolesterol (0.090275), Riwayat Keluarga (0.048127), dan Tekanan Darah (0.025077). Karena ID3 bekerja secara greedy, Usia dipilih sebagai root node.

Langkah 3: Membentuk cabang sampai leaf murni:

- Cabang 'Paruh_Baya': keempat pasien (P03, P07, P12, P13) semuanya Positif, jadi langsung menjadi leaf POSITIF.
- Cabang 'Muda': dicek dengan atribut sisanya. Kolesterol memisahkan cabang ini dengan sempurna (gain = 0.970951, sama dengan entropy cabangnya). Pasien muda dengan Kolesterol Normal (P09, P11) semuanya Positif, sedangkan pasien muda dengan Kolesterol Tinggi (P01, P02, P08) semuanya Negatif.
- Cabang 'Lanjut': semua pasiennya bertekanan darah Normal, jadi atribut itu tidak berguna lagi. Riwayat Keluarga memisahkan cabang ini dengan sempurna. Pasien tanpa riwayat keluarga (P04, P05, P10) Positif, sedangkan yang punya riwayat (P06, P14) Negatif.

Pohon keputusan yang terbentuk:

```
[ Usia ? ]
   ├── Paruh_Baya ──────────────────────────────> [ POSITIF ] (Pure, n=4)
   ├── Muda
   │     └── [ Kolesterol ? ]
   │           ├── Normal ──────────────────────> [ POSITIF ] (Pure, n=2)
   │           └── Tinggi ──────────────────────> [ NEGATIF ] (Pure, n=3)
   └── Lanjut
         └── [ Riwayat_Keluarga ? ]
               ├── Tidak ─────────────────────────> [ POSITIF ] (Pure, n=3)
```

```
                └── Ya ──────────────────────────> [ NEGATIF ] (Pure, n=2)
```

6.3 Perhitungan Manual Naïve Bayes

Langkah 1: Menghitung prior

P(Y = Positif) = 9/14 ≈ 0.642857; P(Y = Negatif) = 5/14 ≈ 0.357143.

Langkah 2: Menghitung likelihood tiap atribut

- P(Usia=Muda | Positif) = 2/9; P(Usia=Muda | Negatif) = 3/5
- P(Tekanan=Tinggi | Positif) = 2/9; P(Tekanan=Tinggi | Negatif) = 2/5
- P(Kolesterol=Normal | Positif) = 5/9; P(Kolesterol=Normal | Negatif) = 1/5
- P(Riwayat=Ya | Positif) = 3/9; P(Riwayat=Ya | Negatif) = 3/5

Langkah 3: Menghitung pasien baru

Pasien baru: x_baru = (Usia = Muda, Tekanan = Tinggi, Kolesterol = Normal, Riwayat = Ya).

Skor yang belum dinormalisasi:

> **Score(Positif) = P(P) * P(Muda|P) * P(Tinggi|P) * P(Normal|P) * P(Ya|P) = (9/14) * (2/9) * (2/9) * (5/9) * (3/9) = 10 / 1701 ≈ 0.00587889**

> **Score(Negatif) = P(N) * P(Muda|N) * P(Tinggi|N) * P(Normal|N) * P(Ya|N) = (5/14) * (3/5) * (2/5) * (1/5) * (3/5) = 9 / 875 ≈ 0.01028571**

Langkah 4: Normalisasi posterior

Evidence P(x_baru) = 0.00587889 + 0.01028571 = 0.01616460.

> **P(Positif | x_baru) = 0.00587889 / 0.01616460 = 36.37%**

> **P(Negatif | x_baru) = 0.01028571 / 0.01616460 = 63.63%**

Keputusan Naïve Bayes: karena P(Negatif | x_baru) = 63.63% lebih besar daripada P(Positif | x_baru) = 36.37%, pasien baru diklasifikasikan sebagai NEGATIF (tidak terindikasi penyakit jantung).

BAB VII
EVALUASI KINERJA MODEL KLASIFIKASI

7.1 Confusion Matrix

Confusion matrix menunjukkan perbandingan antara hasil prediksi model dan label aktual:

Tabel 7.1: Confusion Matrix untuk Klasifikasi Biner

Kondisi	Aktual Positif (Y = 1)	Aktual Negatif (Y = 0)
Prediksi Positif (Y_pred = 1)	True Positive (TP)	False Positive (FP) - Tipe I
Prediksi Negatif (Y_pred = 0)	False Negative (FN) - Tipe II	True Negative (TN)

Tabel ini punya 4 kemungkinan hasil. Pada kasus medis, False Negative (Tipe II) paling berbahaya, karena pasien yang sebenarnya sakit justru dinyatakan sehat dan tidak mendapat penanganan.

7.2 Metrik Evaluasi

1. Accuracy: proporsi prediksi yang benar dari seluruh data, yaitu (TP + TN) / (TP + TN + FP + FN).
2. Precision: dari semua yang diprediksi positif, berapa yang benar-benar positif, yaitu TP / (TP + FP).
3. Recall (Sensitivity): dari semua kasus positif yang sebenarnya, berapa yang berhasil ditemukan, yaitu TP / (TP + FN).
4. Specificity: dari semua kasus negatif yang sebenarnya, berapa yang berhasil dikenali, yaitu TN / (TN + FP).
5. F1-Score: rata-rata harmonik dari precision dan recall, yaitu 2 × (Precision × Recall) / (Precision + Recall).

7.3 Kurva ROC dan AUC

Kurva ROC menggambarkan hubungan antara True Positive Rate (sumbu Y) dan False Positive Rate (sumbu X) pada berbagai threshold. AUC (Area Under

Curve) adalah luas di bawah kurva tersebut. Nilainya biasanya antara 0.5 (sama dengan tebakan acak) sampai 1.0 (klasifikasi sempurna).

BAB VIII
ANALISIS KOMPLEKSITAS DAN KARAKTERISTIK ALGORITMA

8.1 Perbandingan Kompleksitas Big-O

Analisis Big-O penting untuk menentukan apakah sebuah algoritma masih bisa dipakai saat datanya besar:

Tabel 8.1: Kompleksitas Big-O Algoritma Klasifikasi

Algoritma Klasifikasi	Waktu Latih (Training)	Waktu Uji (Inferensi)	Kebutuhan Memori
Decision Tree (CART/C4.5)	O(d * N log N)	O(depth) ≤ O(d)	O(Nodes)
Naïve Bayes (Gaussian)	O(N * d)	O(K * d)	O(K * d)
SVM (Linear Kernel)	O(N^2 * d)	O(d)	O(d)
SVM (RBF Kernel)	O(N^2 * d) s/d O(N^3)	O(n_sv * d)	O(n_sv * d)
Logistic Regression	O(L * N * d)	O(d)	O(d)
Neural Network (MLP)	O(L * N * ∑ W_l)	O(∑ W_l)	O(∑ W_l)

Naïve Bayes punya kompleksitas training paling kecil, yaitu O(N·d), karena cukup menghitung frekuensi. SVM dengan kernel RBF dan Neural Network jauh lebih mahal, tetapi bisa memodelkan batas keputusan yang kompleks pada dimensi tinggi.

8.2 Kelebihan, Kekurangan, dan Skenario Penggunaan

Tabel 8.2: Perbandingan Kelebihan, Kekurangan, dan Skenario Penggunaan Algoritma

Algoritma	Kelebihan Utama	Kelemahan Utama	Rekomendasi Skenario Terbaik
Decision Tree	Interpretabilitas tinggi (white-box), mudah diaudit, tidak butuh normalisasi.	Rentan overfitting, instabil terhadap perubahan data latih kecil.	Sistem pendukung keputusan medis, perbankan, compliance audit.
Naïve Bayes	Komputasi sangat cepat, efisien memori, tangguh pada data kecil & teks.	Asumsi conditional independence sering tidak realistis pada data nyata.	Filter email spam, klasifikasi dokumen teks, sentiment analysis cepat.
SVM	Jaminan optimum global konveks, margin maksimal	Komputasi kuadratik terhadap N, sulit	Bioinformatika / ekspresi gen, pengenalan wajah,

	mencegah overfitting.	menginterpretasikan bobot (black-box).	klasifikasi citra.
Logistic Regression	Menghasilkan probabilitas terkalibrasi, ringan, minim hyperparameter.	Hanya memodelkan batas linier, rentan underfitting jika data non-linier.	Credit scoring perbankan, estimasi risiko churn, baseline benchmark.
Neural Network (MLP)	Aproksimasi fungsi universal non-linier, ekstraksi representasi laten otomatis.	Membutuhkan data masif, boros daya komputasi/GPU, rawan jebakan overfitting.	Computer vision, pengenalan suara, pemodelan data multi-variat rumit.

Dari kelima algoritma ini, kita harus memilih model yang paling sesuai kebutuhan: apakah butuh penjelasan yang mudah dipahami, akurasi tinggi, atau proses komputasi yang cepat.

BAB IX
IMPLEMENTASI PYTHON DENGAN SCIKIT-LEARN

9.1 Kode Program Pipeline Klasifikasi

Kode berikut memakai scikit-learn (Pedregosa et al., 2011) untuk melatih kelima model pada data 14 pasien. Evaluasinya memakai Leave-One-Out Cross-Validation (LOOCV) supaya hasilnya tidak hanya dihitung dari data latih:

```
# ================================================================
# PIPELINE KOMPARASI MODEL KLASIFIKASI DATA MINING (INF11121)
# Nama: Nabil (NIM: 2401020073)
# ================================================================
import warnings
warnings.filterwarnings("ignore")
import numpy as np
import pandas as pd
from sklearn.preprocessing import OrdinalEncoder
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import CategoricalNB
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.model_selection import LeaveOneOut, cross_val_score


data = {
    'Usia': ['Muda','Muda','Paruh_Baya','Lanjut','Lanjut','Lanjut','Paruh_Baya',
             'Muda','Muda','Lanjut','Muda','Paruh_Baya','Paruh_Baya','Lanjut'],
    'Tekanan': ['Tinggi','Tinggi','Tinggi','Normal','Normal','Normal','Normal',
                'Normal','Normal','Normal','Normal','Normal','Tinggi','Normal'],
    'Kolesterol': ['Tinggi','Tinggi','Tinggi','Tinggi','Normal','Normal','Normal',
                   'Tinggi','Normal','Tinggi','Normal','Tinggi','Normal','Tinggi'],
    'Riwayat': ['Tidak','Ya','Tidak','Tidak','Tidak','Ya','Ya',
                'Tidak','Tidak','Tidak','Ya','Ya','Tidak','Ya'],
    'Jantung':
['Negatif','Negatif','Positif','Positif','Positif','Negatif','Positif',

'Negatif','Positif','Positif','Positif','Positif','Positif','Negatif']
}
df = pd.DataFrame(data)
features = ['Usia', 'Tekanan', 'Kolesterol', 'Riwayat']
encoder = OrdinalEncoder()
X_enc = encoder.fit_transform(df[features])
y_enc = np.where(df['Jantung'] == 'Positif', 1, 0)
```

```
models = {
    "Decision Tree (entropy)": DecisionTreeClassifier(criterion='entropy',
random_state=42),
    "Categorical Naive Bayes": CategoricalNB(alpha=1.0),
    "SVM (RBF)": SVC(kernel='rbf', probability=True, random_state=42),
    "Logistic Regression": LogisticRegression(random_state=42),
    "Neural Network (MLP)": MLPClassifier(hidden_layer_sizes=(8, 4), max_iter=1000,
random_state=42)
}

loo = LeaveOneOut()
print("=== EVALUASI LEAVE-ONE-OUT CV ===")
for name, clf in models.items():
    scores = cross_val_score(clf, X_enc, y_enc, cv=loo)
    print(f"{name:<26} | Akurasi LOOCV: {scores.mean()*100:6.2f}%")

print("\n=== AKURASI DATA LATIH (referensi saja) ===")
for name, clf in models.items():
    clf.fit(X_enc, y_enc)
    preds = clf.predict(X_enc)
    print(f"{name:<26} | Akurasi: {accuracy_score(y_enc, preds)*100:6.2f}% | CM
(TN,FP,FN,TP): {confusion_matrix(y_enc, preds).ravel()}")

pasien_baru = encoder.transform([['Muda', 'Tinggi', 'Normal', 'Ya']])
print("\n=== INFERENSI PASIEN BARU [Muda, Tinggi, Normal, Ya] ===")
for name, clf in models.items():
    pred = clf.predict(pasien_baru)[0]
    print(f"{name:<26} -> {'POSITIF' if pred == 1 else 'NEGATIF'}")
```

Hasil LOOCV menunjukkan Decision Tree dan MLP paling tinggi (71.43%), diikuti Naïve Bayes dan SVM (64.29%), lalu Logistic Regression (50.00%). Akurasi pada data latih terlihat jauh lebih tinggi (Decision Tree 100%), tapi itu tidak menggambarkan kemampuan generalisasi. Untuk pasien baru [Muda, Tekanan Tinggi, Kolesterol Normal, Riwayat Ya], Naïve Bayes, Decision Tree, dan MLP memprediksi NEGATIF, sedangkan SVM dan Logistic Regression memprediksi POSITIF. Hasil Naïve Bayes dari kode searah dengan perhitungan manual. Decision Tree sklearn tidak sama dengan pohon ID3 manual, yang memprediksi POSITIF lewat cabang Muda lalu Kolesterol Normal. Perbedaan ini

wajar, karena sklearn melakukan threshold split pada encoding ordinal, bukan split kategorikal seperti ID3.

BAB X
KESIMPULAN

Dari pembahasan di atas, ada beberapa hal yang bisa saya simpulkan:

1. Teori dan praktik saling melengkapi. Klasifikasi tidak cukup hanya dengan menjalankan library. Kita perlu paham fungsi objektif, batas keputusan, dan bias tiap algoritma supaya bisa memilih model yang tepat.
2. Interpretabilitas vs kapasitas model. Ada trade-off antara model yang mudah dijelaskan, seperti decision tree, dan model yang lebih kompleks tapi sulit dijelaskan, seperti SVM RBF dan MLP.
3. Metrik evaluasi. Pada kasus berisiko tinggi seperti kesehatan, akurasi saja tidak cukup. Recall, precision, F1-score, dan ROC-AUC perlu diperhatikan untuk menekan False Negative. Simulasi ini juga menunjukkan bahwa akurasi pada data latih bisa menyesatkan, jadi evaluasi LOOCV perlu dipakai.

DAFTAR PUSTAKA

Breiman, L., Friedman, J., Olshen, R. A., & Stone, C. J. (1984). Classification and Regression Trees. Belmont, CA: Wadsworth International Group.

Cortes, C., & Vapnik, V. (1995). Support-vector networks. Machine Learning, 20(3), 273–297.

Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. Mathematics of Control, Signals and Systems, 2(4), 303–314.

Han, J., Kamber, M., & Pei, J. (2012). Data Mining: Concepts and Techniques (3rd ed.). Waltham: Morgan Kaufmann Publishers.

Hunt, E. B., Marin, J., & Stone, P. J. (1966). Experiments in Induction. New York: Academic Press.

Minsky, M., & Papert, S. A. (1969). Perceptrons: An Introduction to Computational Geometry. Cambridge: MIT Press.

Mitchell, T. M. (1997). Machine Learning. New York: McGraw-Hill Education.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

Quinlan, J. R. (1986). Induction of decision trees. Machine Learning, 1(1), 81–106.

Quinlan, J. R. (1993). C4.5: Programs for Machine Learning. San Mateo: Morgan Kaufmann Publishers.

Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. Psychological Review, 65(6), 386–408.

Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. Nature, 323(6088), 533–536.

Shannon, C. E. (1948). A mathematical theory of communication. Bell System Technical Journal, 27(3), 379–423.

Tan, P.-N., Steinbach, M., Karpatne, A., & Kumar, V. (2019). Introduction to Data Mining (2nd ed.). New York: Pearson Education.

Wolpert, D. H., & Macready, W. G. (1997). No free lunch theorems for optimization. IEEE Transactions on Evolutionary Computation, 1(1), 67–82.